
Auditable Rust/CUDA Systems for Parameter-Constrained Language Model Training

Cedric Haddad

Department of Computer Science
University of Chicago
Chicago, IL 60637

Abstract

OpenAI’s Parameter Golf challenge asks participants to train the lowest bits-per-byte (BPB) language model whose code and compressed weights fit in a 16 MB artifact and whose train and evaluation phases each fit within 600 seconds on $8\times H100$ GPUs. This setting turns implementation details—precision flow, memory traffic, data residency, graph capture, optimizer communication, and artifact accounting—into first-class modeling constraints. We present the current state of a Rust/CUDA training stack for this challenge. Unlike the original project proposal, this paper does not claim a leaderboard-winning BPB or a completed quantization macro compiler. Instead, we report a systems case study: after correcting the record-shaped training surface and moving from an F32-centered implementation with BF16 islands to an audited BF16 direct-compact profile, our current $8\times H100$ record-shaped profile reaches a mean of 116.96 ms/step at the current frontier-derived training shape of 1024 context tokens and 524,288 global tokens per step. We describe the Rust workspace, the spec-owned runtime profiles, the BF16/cuDNN attention path, the device-resident token schedule, CUDA graph capture, and sharded Muon graph paths that produced this result. We also document unresolved record-readiness gaps: no full 600-second train/eval/export run has yet been completed, BPB is not yet validated on a meaningful full-validation run, the final code+artifact byte budget is still open, NCCL overlap is currently disabled, and the proposal’s quantization procedural macros and novel XSA/BigramHash fusions remain future work. The main contribution of this draft is an audit-first account of what has and has not been implemented, and a decomposition of the remaining path from a fast Rust systems prototype to a valid Parameter Golf record submission.

1 Introduction

Modern language-model training stacks are overwhelmingly built on Python frameworks such as PyTorch and JAX. For most research workflows, this is the right tradeoff: automatic differentiation, mature kernels, package ecosystems, and fast iteration outweigh the overhead of a dynamic host language. Parameter Golf [1], however, is an unusually tight systems benchmark. The challenge evaluates language models by bits-per-byte (BPB) on FineWeb validation data, while constraining submissions to a 16,000,000 byte decimal artifact, including code and compressed model weights, and to 600 seconds for training and 600 seconds for evaluation on $8\times H100$ GPUs.

These constraints make several normally secondary engineering choices central. A run that wastes 100 ms per step may lose thousands of optimizer updates. A model that materializes a full [tokens, vocab] logits tensor may exceed the memory-traffic budget even if it fits in HBM. A legal test-time training (TTT) implementation must score validation tokens before updating on them. A tokenizer or byte-

sidecar mismatch can produce an apparently good BPB that is not legally comparable. A compressed model that fits just under 16 MB can still fail if the code bytes are not counted.

This project began from the hypothesis that a Rust/CUDA implementation could both recover throughput and enable implementation-level design choices that are difficult to express in Python/PyTorch. The original proposal emphasized three speculative directions: a procedural-macro quantization-scheme compiler, custom kernel fusions such as XSA-in-attention and BigramHash-embedding merge, and compile-time architecture specialization. The current codebase has not yet completed those speculative research pieces. Instead, the project has first become a study in reconstructing a frontier-shaped training surface in Rust with explicit auditability. That pivot is important: before we can claim that Rust enables new modeling ideas, we must first show that the Rust stack can faithfully and efficiently execute the same computational graph as the public frontier.

The paper therefore asks a narrower and more measurable question:

What systems work is required for a Rust/CUDA implementation to match a current-frontier Parameter Golf training surface, and which gaps remain before it can claim a valid leaderboard score?

Our current answer is mixed. On the systems side, we have reached a record-shaped $8\times H100$ profile at 116.96 ms/step for the current audited training shape. This closes a large gap from earlier substantiated Rust profiles around 256.8 ms/step and from an even earlier invalid/overheavy path that exceeded tens of seconds per step. On the record side, we do not yet claim a BPB result: a full train/eval/export run has not completed, the artifact budget is not yet final, and 3-seed full-validation statistics are absent.

2 Background: Parameter Golf and Frontier Drift

Parameter Golf is an optimization problem over both model quality and systems constraints. The official challenge asks for the best BPB under a fixed model-artifact size and fixed train/eval time. The score is tokenizer-agnostic because cross-entropy is normalized by byte counts, not token counts alone. In practice, the competition rapidly became a joint search over architecture, optimizer, quantization, evaluation-time adaptation, and implementation.

The public frontier moved substantially during the project. Early versions of this proposal assumed an approximately 1.12 BPB state of the art based on 1024-vocabulary XSA-era submissions. Later public records moved to SP8192/CaseOps tokenization, recurrent depth, parallel residuals, sparse attention gates, LQER quantization recovery, per-group compression, and phased score-first TTT. For example, PR #1855 reported a 3-seed mean of 1.06108 BPB, a mean of about 121.7 ms/step, 4917–4945 training steps in 600 seconds, artifacts under 16 MB, and evaluation under 600 seconds [2]. This changed the appropriate comparison target: the Rust stack should not be measured against the early 1.12 BPB frontier, but against a later current-frontier shape.

The current audited Rust target therefore follows a PR #2135-derived shape rather than the older 2048-token, 786,432-token/step target. The primary audited target uses 1024 training context, 2560 evaluation/TTT context, 524,288 global train tokens per step, 8 GPUs, 64 local microbatches per rank, a BF16 direct-compact backward profile, device-resident token scheduling, and a no-loss CUDA graph record profile. NCCL overlap is currently disabled in the audited profile. The corresponding TOML spec is kept in the repository as `frontier_2135_audit_target.toml` [3].

3 System Design

3.1 Rust workspace

The implementation is organized as a Rust workspace with crates corresponding to the main systems components:

- `pg-core`: GPU tensor wrappers, CUDA/NCCL utilities, and low-level error handling.
- `pg-kernels`: CUDA and cuDNN/cuBLAS-facing kernels, including BF16 SDPA wrappers and fused pointwise kernels.

- `pg-model`: model configuration, forward/backward orchestration, explicit GPU activation storage, and runtime profiles.
- `pg-optim`: GPU AdamW and Muon update paths, including sharded Parallel Muon support.
- `pg-data`: token-shard and byte-sidecar loading.
- `pg-quant`: quantized artifact export/import, LQER metadata, and compression choices.
- `pg-eval`: sliding evaluation and GPU LoRA/phased TTT infrastructure.
- `pg-train`: training entry point, record-shaped benchmarking, audit logs, and Modal deployment integration.

This organization is not only a software-engineering convenience. It lets the training run own an explicit *spec* describing architecture, precision, output-CE backend, distributed optimizer backend, evaluation backend, runtime profile, and unsupported-variant rules. The current `RunSpec` includes explicit enums for attention backend, compute precision, output CE backend, distributed optimizer backend, eval adaptation backend, backward chain profile, CUDA graph profile, NCCL overlap mode, and record profile [4]. This is a central lesson of the project: in a challenge where environment-variable toggles can silently change the hot path, record-shaped runs need spec-owned runtime profiles rather than informal shell commands.

3.2 Current frontier-shaped spec

The current audited target is named `frontier_2135_audit_target`. Its model family is `frontier_1855_like`, with 8192 vocabulary, 11 layers, model dimension 512, 8 query heads, 4 KV heads, $4\times$ MLP expansion, partial RoPE, U-Net skips, recurrence over mid layers, split attention/MLP parallel residuals, CaseOps byte-sidecar accounting, SparseAttnGate with scale 0.5, BOS-aware SmearGate, and asymmetric logit softcaps. Its training shape is:

Quantity	Value
Training context	1024 tokens
Evaluation/TTT context	2560 tokens
Global train tokens/step	524,288
World size	8 H100 GPUs
Local microbatches/rank	$524,288 / (8 \cdot 1024) = 64$
Distributed optimizer	Sharded Parallel Muon
Runtime profile	<code>frontier2135_audit</code>
Backward chain	<code>bf16_direct_compact</code>
CUDA graph profile	<code>record_step_no_loss</code>
NCCL overlap	off

This corrected shape matters. Earlier 2048-context, 786,432-token/step profiles measured a heavier training surface and should not be treated as the primary target inherited from the current public frontier. Using the wrong target shape is a systems bug: it can make a correct implementation look slower than the reference for reasons unrelated to language choice or kernel quality.

3.3 BF16 direct-compact execution

The initial Rust GPU stack followed a common migration path: F32 storage and residual buffers with BF16 islands around GEMMs or attention. This was useful for correctness but left multiple F32 $\leftrightarrow$ BF16 bridges in the hot path. The current audited profile instead exposes a BF16 direct-compact backward chain and records the absence of F32 hot-path bridges in the audit. In this mode, the model uses cuDNN BF16 SDPA, fused QKV/RoPE/prepack forward, chunked BF16 output CE, and compact BF16 caches where the runtime profile allows them. The output CE backend is currently `chunked_bf16_cache`: it avoids a persistent full local-batch logits tensor, but it is not yet the final exact fused WMMA output-projection/softcapped-CE kernel.

3.4 Data residency and CUDA graph capture

A previous record-shaped path still relied on host batch copies and CPU-owned token buffers. The current audited profile uses a GPU token-ring full-schedule sampler, eliminating hot-path host batch copies in record-shaped timing. This change is important not only for bandwidth but for CUDA graph capture: changing host pointers or synchronizing on host batch construction prevents a stable captured graph. The current profile captures a no-loss record-step graph and enables side GEMM capture, allowing the measured path to exercise a stable, record-shaped GPU schedule.

3.5 Optimizer graphing and sharded Muon

The audited profile uses sharded Parallel Muon with graph paths for local and pre-norm work. This is distinct from the earlier all-reduce-replicated Muon path, where every rank redundantly updated full parameter banks. The current profile keeps NCCL overlap off; communication/computation overlap remains unproven and is listed as future work. Even so, sharded optimizer graphing was a necessary step toward frontier-shaped performance.

4 Methodology

4.1 Audit-first execution

The project uses an audit-first methodology. Each record-shaped run prints not only wall-clock timing but also the training shape, attention backend, precision profile, output CE backend, distributed optimizer backend, local microbatch count, token schedule mode, CUDA graph profile, artifact byte fields, and CaseOps byte-sidecar hash when available. The intent is to make it impossible for a run to appear record-shaped while silently falling back to a debug kernel, CPU data path, synthetic data, wrong batch shape, or model-only artifact budget.

This approach was motivated by several bugs discovered during development:

- an earlier target shape used the wrong context/batch-token pair for the inherited frontier comparison;
- alignment-unsafe GPU/shard download paths needed correction;
- GPU LoRA eval required a no-full-logit chunked BF16 CE path to match the training/export assumptions;
- the artifact pipeline revealed an over-budget proxy artifact before a mixed int5/int6 export fix;
- runtime flags were too easy to use in combinations that produced stale or mismatched hot-path buffers.

These are not incidental engineering problems. In Parameter Golf, they are exactly the kinds of issues that can lead to invalid scores.

4.2 Measurement target

The timing results in this draft are record-shaped systems measurements, not final record submissions. The primary profile uses the corrected `frontier_2135_audit_target` shape described above. We report H100 record-shaped timing only for runs that preserve the current audited training shape. We do not report full-validation BPB, because no complete full train/eval/export run has yet been validated.

4.3 What is not measured yet

This draft deliberately omits several quantities that would be required for a final record paper:

- full 600-second training run with artifact export;
- full validation BPB with CaseOps byte-sidecar accounting;

- evaluation wall-clock under the 600-second cap;
- final code+compressed-model byte proof under 16,000,000 bytes;
- three-seed mean and standard deviation;
- ablations for the proposal’s novel XSA-in-attention, BigramHash-embedding, and quantization-macro components.

5 Current Results

5.1 Record-shaped throughput

Table 1 shows the current best audited $8\times H100$ record-shaped timing. All three runs use the same corrected shape: 1024 training context and 524,288 global tokens per step. The median step time is 117.036 ms and the mean is 116.963 ms.

Table 1: Current audited $8\times H100$ record-shaped profile. These are systems timings, not full record BPB runs.

Run	Shape	Measured ms/step
frontier2135_official1024_graph_opt48.json	$T = 1024, 524,288$ tokens/step	117.044
frontier2135_official1024_graph_opt48_r2.json	same	116.808
frontier2135_official1024_graph_opt48_r3.json	same	117.036
Mean	same	116.963
Median	same	117.036

At 524,288 global tokens per step, a 116.963 ms profile corresponds to approximately 4.48 million global tokens/second. If this profile generalized to a full train run without added export/eval overhead, it would allow roughly 5129 steps in 600 seconds. We do not claim that extrapolation as a final result; the current profile is explicitly a record-shaped systems profile with a no-loss CUDA graph, not a full end-to-end record submission.

5.2 Implemented systems components

The following components are implemented and have been exercised in the current profile or related proxy/parity runs:

- current-frontier TOML specs, including `frontier_2014_clean_target.toml` and `frontier_2135_audit_target.toml`;
- corrected frontier train shape of 1024 context and 524,288 train tokens per step;
- spec-visible BF16 direct-compact backward profile;
- active BF16 backward chain with zero audited F32 hot-path bridge count;
- GPU token-ring full-schedule sampler;
- elimination of hot-path host batch copies in record-shaped timing;
- no-loss CUDA graph capture for the record-shaped step;
- sharded Parallel Muon local/pre-norm graph paths;
- fused QKV/RoPE/prepack forward path;
- CaseOps sidecar SHA emission;
- artifact byte/hash audit plumbing exercised in proxy;
- fixed alignment-unsafe GPU/shard download paths;
- GPU LoRA eval support for chunked BF16 CE without materialized full logits;
- corrected MixedInt5Int6 export so MLP tensors are actually int5 rather than accidentally int6.

5.3 Not yet implemented or not yet validated

The current stack remains short of record readiness. The following items are not yet complete:

- no full record train/eval/export run has been completed;
- BPB has not been validated on a meaningful full-validation run;
- the artifact budget is still open: the last completed proxy before the int5 MLP export fix was over budget at 16,578,935 bytes, and the corrected run has not yet completed;
- NCCL overlap is unproven and currently disabled;
- persistent-CTA megakernel backward fusion is not implemented;
- the original proposal’s quantization procedural-macro compiler is not implemented;
- the original proposal’s BigramHash embedding merge is not implemented;
- the original proposal’s XSA-in-attention fusion is not implemented as described.

6 Analysis

6.1 What produced the current systems gain?

The current throughput improvement is primarily a systems-engineering result, not an algorithmic novelty result. The key improvements were:

Correct target shape. The inherited public-frontier training shape uses 1024 context and 524,288 global tokens per step. Treating a 2048-context, 786,432-token shape as the primary target overstated the gap and measured a different workload. Fixing the spec made the benchmark comparable.

Spec-owned profiles. The codebase now exposes runtime-critical choices in RunSpec: attention backend, compute precision, output CE backend, distributed optimizer backend, eval adaptation backend, backward chain profile, CUDA graph profile, NCCL overlap mode, and record profile. This prevents record-shaped runs from depending solely on environment-variable combinations that are difficult to audit.

BF16 direct-compact chain. The current profile replaces an F32-centered training graph with a BF16 direct-compact backward chain. The earlier BF16 implementation had BF16 islands inside an F32 graph; the current profile explicitly audits the hot-path bridge count.

Device-resident token schedule. The GPU token-ring full-schedule sampler removes hot-path host batch copies. This reduces host synchronization and enables stable graph capture.

CUDA graph capture. The record-step no-loss graph captures a fixed-shape hot path. Since the Parameter Golf train shape is fixed by the record spec, this is a natural match for CUDA graph execution.

Sharded optimizer graphing. Sharded Parallel Muon graph paths reduce redundant optimizer work relative to replicated update paths, though NCCL overlap is still disabled.

6.2 What did not produce the current gain?

The current result should not be attributed to the speculative contributions in the original proposal. The quantization procedural-macro compiler is not present. XSA-in-attention and BigramHash-embedding fusion are not implemented as proposed. The current throughput improvement instead comes from precision-flow cleanup, graph capture, data residency, target-shape correction, and optimizer/runtime auditability.

This is a useful negative result: at the current stage, Rust’s value has not been “automatic speed from avoiding Python.” Rust’s value has been stronger control over the training surface and the ability to encode correctness/compliance invariants directly in the spec and runtime.

7 Compliance and Validity

A Parameter Golf systems paper must distinguish three levels of validity.

Systems-valid timing. A timing is systems-valid if it uses the intended record-shaped batch arithmetic, training context, global batch tokens, backend choices, and GPU schedule. The 116.96 ms/step result is valid at this level for the audited profile.

Record-valid run. A run is record-valid only if the full training phase completes under 600 seconds, the full evaluation phase completes under 600 seconds, no validation tokens are used before scoring, code plus compressed model fit under 16,000,000 bytes, and BPB is computed on the official validation bytes. We do not yet claim record validity.

Leaderboard-valid result. A leaderboard result additionally requires full logs, reproducibility, and sufficient seed statistics. The current draft does not claim leaderboard validity or a final BPB.

8 Relation to the Original Proposal

The original proposal argued for three Rust-enabled research directions: a quantization-scheme compiler via procedural macros, novel kernel fusions beyond `torch.compile`, and compile-time architecture specialization. Those ideas remain future work. The current implementation instead focused on making the training surface auditable and fast enough to make such experiments meaningful.

This changes the paper’s contribution. Rather than claiming that the proposal’s novel fusions improve BPB, the current draft claims that:

1. a Rust/CUDA stack can reproduce a current-frontier-shaped training surface with explicit auditability;
2. spec-owned runtime profiles prevent silent hot-path changes;
3. BF16 direct-compact execution, device-resident data, CUDA graph capture, and optimizer graphing close a large systems gap;
4. audit hardening found real correctness and compliance bugs;
5. several original proposal components remain unimplemented and should be evaluated only after a full record-valid baseline exists.

9 Future Work

Full record run and BPB validation. The immediate next step is to complete a full train/eval/export run with the corrected artifact path, CaseOps byte-sidecar BPB, legal score-first TTT audit, and code+model byte accounting.

Artifact budget closure. The artifact path must close below 16,000,000 bytes after the int5 MLP export fix and any code bytes are counted. The current draft cannot claim artifact compliance.

NCCL overlap. The current audited profile disables NCCL overlap. Communication overlap remains an important next target for improving robustness and recovering headroom for full-loss/eval overhead.

Persistent-CTA backward fusion. Persistent-CTA or megakernel backward fusion is not implemented. It may reduce launch overhead and improve cache locality in the compact BF16 backward chain.

Original proposal fusions. The original XSA-in-attention, BigramHash-embedding merge, and fused RMSNorm/QK/RoPE/q-gain ideas should be tested as controlled ablations after the full baseline is record-valid. The correct ablation is not “Rust vs. PyTorch” in the abstract, but “frontier-equivalent Rust baseline vs. Rust baseline plus one fusion.”

Quantization macro compiler. The procedural-macro quantization compiler remains an interesting research direction, but it should be evaluated after the artifact exporter and BPB pipeline are stable. Otherwise, quantization experiments risk confounding compression bugs with true model-quality effects.

10 Limitations

This draft has several important limitations. First, it reports no final BPB and no three-seed statistics. Second, the primary timing uses a no-loss CUDA graph record-shaped profile, so it should not be confused with a full train/eval/export submission. Third, the artifact budget remains unresolved. Fourth, the current codebase inherits many algorithmic choices from public frontier submissions rather than introducing a new model architecture. Fifth, several proposal components are not implemented. Finally, the current results are from internal project logs and must be reproduced as part of a public artifact before being treated as definitive.

11 Conclusion

We presented the current state of a Rust/CUDA Parameter Golf training stack. The main result is not a leaderboard BPB, but an audited systems milestone: a current-frontier-shaped $8\times H100$ profile at 116.96 ms/step for 1024 context and 524,288 global tokens per step. The path to this milestone required correcting the target shape, making runtime profiles spec-owned, moving to a BF16 direct-compact chain, eliminating hot-path host batch copies, capturing a no-loss CUDA graph, and graphing sharded Muon paths. The remaining work is substantial: a full record train/eval/export run, official BPB validation, artifact budget closure, NCCL overlap, and controlled ablations for the original proposal’s novel fusions and quantization compiler. The broader lesson is that systems-level languages are not automatically faster; their advantage in this setting comes from making the computational graph, precision flow, data residency, and compliance constraints explicit enough to audit and optimize.

References

- [1] OpenAI. Parameter Golf: Train the smallest LM you can that fits in 16MB. <https://github.com/openai/parameter-golf>, 2026.
- [2] Codemath3000. Record: SP8192 + LQER + Sparse Attn Gate + BOS-Fixed SmearGate + 9-Hparam Greedy Stack — val_bpb 1.06108. Pull request #1855, <https://github.com/openai/parameter-golf/pull/1855>, 2026.
- [3] Cedric Haddad. `frontier_2135_audit_target.toml`. https://github.com/cedrichaddad/parameter-golf/blob/main/parameter-golf-rs/specs/frontier_2135_audit_target.toml, 2026.
- [4] Cedric Haddad. Rust Parameter Golf run specification types. <https://github.com/cedrichaddad/parameter-golf/blob/main/parameter-golf-rs/crates/pg-model/src/spec.rs>, 2026.
- [5] K. Jordan. Muon: An optimizer for hidden layers in neural networks. <https://kellerjordan.github.io/posts/muon/>, 2024.
- [6] A. Karpathy. `llm.c`: LLM training in simple, raw C/CUDA. <https://github.com/karpathy/llm.c>, 2024.
- [7] K. Jordan. `modded-nanogpt`: NanoGPT training speedruns. <https://github.com/KellerJordan/modded-nanogpt>, 2024.
- [8] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers. *Proceedings of ICLR*, 2023.
- [9] C. Lowman. `cudarc`: Safe Rust wrapper around the CUDA toolkit. <https://github.com/coreylowman/cudarc>, 2024.
- [10] T. Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691*, 2023.

- [11] NVIDIA. CUDA Graphs. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cuda-graphs>, 2026.